Proposed Pull Requests

Fork `stevenrwood/ioSender` → upstream `terjeio/ioSender` · base branch `master` · 22 branches (PR 8 stacks on PR 3; PRs 9-10 on the integrated keymap/Xbox base; PRs 11-15 off master; PRs 16-17 stack on PR 15; PRs 18-22 off master) · all branches LF-normalized (`.gitattributes * text=auto eol=lf`)

1	RP.Math build fix	`pr/fix-rp-math-build`	1 file · +6
2	Surface Motor Fault / Warning signals	`pr/motor-signals`	1 file · +2/−30
3	Console persistence + Key Mappings editor	`pr/keymapping_editor`	14 files · +1209/−13
4	Connection-loss handling + auto-reconnect	`pr/connection-loss-reconnect`	9 files · +519/−61
5	Stepper calibration (scratch) tab	`pr/stepper-calibration-scratch`	5 files · +589
6	Load Folder + Fusion batch-post add-in	`pr/load-folder`	18 files · +1245/−6
7	High-DPI / large-text layout	`pr/highdpi-layout`	26 files · +123/−107
8	Xbox controller + Key Mappings editor polish	`pr/xbox-mapping`	9 files · +1323/−26
9	Configurable main page + flyouts + jog panels (XL)	`pr/main-page-layout`	29 files · +1467/−52
10	Macro manager + run-only flyout	`pr/macro-editor`	11 files · +464/−26
11	Macro Processor Enhancements	`pr/macro-processor`	9 files · +659/−18
12	Onboarding tooltips	`pr/onboarding-tooltips`	6 files · +83/−24
13	Build documentation (Mega-XL notes + tool-change guide)	`pr/build-docs`	4 files · +1259
14	Combined SD + LittleFS file browser	`pr/sdcard-filesystems`	6 files · +388/−19
15	USB / Ethernet / simulator connection support + Connect menu	`pr/connect-simulator`	16 files · +779/−62
16	Validate controller (check-mode g-code conformance test)	`pr/validate`	4 files · +1176/−37
17	Restore main window position across launches	`pr/window-restore`	2 files · +60/−7
18	grbl:Settings — highlight changed settings + revert	`pr/grbl-settings`	3 files · +137/−7
19	Console — inline terminal-style command prompt	`pr/mdi-console`	6 files · +131/−19
20	3D view — Ctrl+click to jog + per-axis orientation	`pr/click-to-jog`	5 files · +206/−60
21	grbl:Settings — Machine Setup Wizard	`pr/machine-setup-wizard`	8 files · +1194/−1

22	Keyboard jog — keep active when Job-view focus drifts	`pr/keyboard-jog-focus`	2 files · +23/−1

PR 1 Reference RP.Math in CNC Core to fix GCodeEmulator build

Branch	`pr/fix-rp-math-build`
Compare	https://github.com/terjeio/ioSender/compare/master...stevenrwood:ioSender:pr/fix-rp-math-build

File	+	−
`CNC Core/CNC Core/CNC Core.csproj`	6	0

CNC Core compiles `GCodeEmulator.cs`, which uses the `RP.Math.Vector3` type, but the project file declared no reference to RP.Math. A clean build of CNC Core therefore fails to resolve the type.

This adds the `RP.Math` and `RP.Math.Vector3` references using the *same* `HintPath` convention already used elsewhere in the solution — `CNC Controls.csproj` already references `RP.Math.Vector3` via `..\..\Vector-master\RP.Math.Vector3\bin\Release\`. The fix is six lines and purely additive; no code changes.

PR 2 Surface Motor Fault and Motor Warning signals in Signals display

Branch	`pr/motor-signals`
Compare	https://github.com/terjeio/ioSender/compare/master...stevenrwood:ioSender:pr/motor-signals

File	+	−
`CNC Controls/CNC Controls/SignalsControl.xaml`	2	30

Removes the capability-gating `Styles` that prevented the Motor Fault (**F**) and Motor Warning (**W**) indicators from ever showing.

grblHAL does not advertise these as `NEWOPT` capability tokens, so the visibility gates were effectively dead code — the LEDs were permanently hidden. Motor Fault / Warning are now always displayed alongside the other signal LEDs (H / S / P), turning red when the corresponding signal triggers. This surfaces the Motor Fault signal as intended, without requiring the firmware to advertise a capability flag.

PR 3 Persist console state and add an in-app Key Mappings editor

Branch	`pr/keymapping_editor`
Compare	https://github.com/terjeio/ioSender/compare/master...stevenrwood:ioSender:pr/keymapping_editor

File	+	−
CNC Controls/CNC Controls/AppConfig.cs	14	0
CNC Controls/CNC Controls/AppConfigView.xaml	1	1
CNC Controls/CNC Controls/AppConfigView.xaml.cs	16	3
CNC Controls/CNC Controls/CNC Controls.csproj	7	0
CNC Controls/CNC Controls/ConsoleControl.xaml	3	3
CNC Controls/CNC Controls/ConsoleControl.xaml.cs	14	0
CNC Controls/CNC Controls/ConsoleWindow.xaml.cs	47	0
CNC Controls/CNC Controls/KeyMapEditor.xaml	115	0
CNC Controls/CNC Controls/KeyMapEditor.xaml.cs	606	0
CNC Core/CNC Core/CNC Core.csproj	1	0
CNC Core/CNC Core/KeypressHandler.cs	111	0
CNC Core/CNC Core/ShortcutKey.cs	136	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	69	3
ioSender/ioSender/MainWindow.xaml.cs	69	3

Persists the console's state across sessions and replaces the old App Settings "*Save key mappings*" button (which only dumped the current bindings to `KeyMap<mode>.xml` for hand-editing) with a proper in-app **Edit Key Mappings** dialog. Applies to both **ioSender** and **ioSender XL**.

Console persistence: the three console checkboxes (*Verbose*, *Filter out realtime responses*, *Show all realtime responses*), whether the floating console window is open, and that window's size/position (clamped to the visible desktop so it can never restore off-screen) are all remembered. A new `ConsoleShortcut` setting (default `Esc`) toggles the console; it is now a first-class action inside the mappings editor rather than a separate, ad-hoc tab. A shared `ShortcutKey` helper (parse / storage / display) is used by both main windows, and an `AppConfig.ConsoleShortcutChanged` event re-registers the key live without a restart.

Key Mappings editor (`KeyMapEditor`, modal so jog/realtime dispatch can't fire mid-capture):

- One grouped, collapsible outline of every bindable action (grouped by function, like the Load Folder outline; groups remember their expansion per session). Click a row, press a combo to assign; right-click for *Reset to default* / *Clear*.
- Bindings changed from the factory default are highlighted; group headers flag a modified group and offer *Expand all / Collapse all / Reset group*.
- Per-axis jog keys are folded into the same list; rows and group headers carry tooltips.
- A *No-numpad keys* preset remaps the NumPad jog/step/feed actions to `Ctrl+1–8` and `[ ] − =` for keyboards without a numeric keypad (Windows exposes no reliable API to detect numpad presence, so this is a user-triggered preset).

The `KeypressHandler` gains an editor API (`GetActionBindings`/`GetJogBindings`/`Apply…`) over its existing function catalog, identifying bindings by reflection method name with a friendly-label map. Persistence still uses the existing `SaveMappings`/`LoadMappings`.

PR 4 Fix unhandled exceptions on connection loss (e.g. SD card swap)

Branch	`pr/connection-loss-reconnect`
Compare	https://github.com/terjeio/ioSender/compare/master...stevenrwood:ioSender:pr/connection-loss-reconnect

File	+	−
CNC Controls/CNC Controls/AppConfig.cs	5	0
CNC Controls/CNC Controls/SDCardView.xaml.cs	13	0
CNC Core/CNC Core/Comms.cs	74	0
CNC Core/CNC Core/EltimaStream.cs	6	0
CNC Core/CNC Core/Grbl.cs	13	4
CNC Core/CNC Core/GrblViewModel.cs	45	0
CNC Core/CNC Core/SerialStream.cs	179	42
CNC Core/CNC Core/TelnetStream.cs	105	7
CNC Core/CNC Core/WebsocketStream.cs	79	8

When the controller's serial/USB device disappears — for example a board that resets and re-enumerates when its SD card is removed or reinserted — the status-poll timer kept writing to the dead port. The write threw (an `IOException`, then an `InvalidOperationException`: "BaseStream only available when the port is open") on a `System.Timers.Timer` thread, and the unhandled exception terminated the whole process. Opening the SD card tab afterwards threw similarly from `PurgeQueue`.

Changes:

- Guard all stream writes (`IsOpen` + try/catch). A device-gone fault now closes the port and starts an auto-reconnect instead of crashing.
- Add a shared, transport-agnostic `Reconnector` (used by serial, network and websocket) exposing `ConnectionLost` / `Reconnected` events. `GrblViewModel` handles them (stop/resume polling, show status); it is wired up at connect time in `AppConfig`.
- **Serial:** refactor open into a reusable `OpenPort()` that probes `GetPortNames()` and re-opens; publish the port only once it is actually open, to avoid a non-null-but-closed race; make `PurgeQueue` tolerant of a closed port; harden the receive handler against a port torn down mid-read.
- **Telnet / websocket:** guarded writes, loss detection (read returning 0 / `OnClose`) and reconnect via the same helper.
- Wrap the poll-timer callback in try/catch as a last line of defence.
- Guard the SD card view against acting on a closed connection.

PR 5 Add Stepper calibration (scratch) tab to grbl:Settings

Branch `pr/stepper-calibration-scratch`

Compare https://github.com/terjeio/ioSender/compare/master...stevenrwood:ioSender:pr/stepper-calibration-scratch

File	+	−
`CNC Controls/CNC Controls/CNC Controls.csproj`	7	0
`CNC Controls/CNC Controls/GrblConfigView.xaml`	3	0
`CNC Controls/CNC Controls/IGrblConfigTab.cs`	1	0
`CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml`	92	0
`CNC Controls/CNC Controls/StepperCalibrationScratchWizard.xaml.cs`	486	0

Adds a new tab to grbl:Settings for V-bit *scratch-line* steps/mm calibration over a long span — a more accurate alternative to the short jog-based calibration for the X and Y axes.

It generates an `.nc` program that scratches paired lines (perpendicular to the calibrated axis) for several candidate steps/mm values. Because GRBL cannot change `$100`/`$101` mid-program, each candidate is simulated using the *commanded-distance trick*: the commanded distance `C = span × candidate / current`, so the machine physically travels the distance that the candidate steps/mm value *would* produce.

The program is loaded into the job view (with an optional save to disk). After cutting, the user measures the actual spacings; the tab then computes and saves the corrected `$100`/`$101` from those measurements (an implied value per pair, averaged). The prolog mirrors the Load Folder convention. X/Y only — Z continues to use the existing jog wizard.

Includes a small follow-up commit with high-DPI layout tweaks for the new tab.

PR 6 Add "Load Folder": load a folder of per-toolpath .nc files as one outlined program

Branch	`pr/load-folder`
Compare	https://github.com/terjeio/ioSender/compare/master...stevenrwood:ioSender:pr/load-folder

File	+	−
CNC Controls/CNC Controls/CNC Controls.csproj	2	0
CNC Controls/CNC Controls/FolderPicker.cs	159	0
CNC Controls/CNC Controls/FusionFolderLoader.cs	313	0
CNC Controls/CNC Controls/GCode.cs	109	0
CNC Controls/CNC Controls/GCodeListControl.xaml	37	0
CNC Controls/CNC Controls/GCodeListControl.xaml.cs	84	0
CNC Controls/CNC Controls/JobControl.xaml.cs	4	0
CNC Core/CNC Core/GCodeJob.cs	41	5
CNC Core/CNC Core/GrblViewModel.cs	7	1
Fusion Addin/README.md	95	0
Fusion Addin/install-macos.sh	39	0
Fusion Addin/install-windows.ps1	40	0
Fusion Addin/ioSenderBatchPost/ioSenderBatchPost.manifest	11	0
Fusion Addin/ioSenderBatchPost/ioSenderBatchPost.py	292	0
ioSender XL/ioSender XL/MainWindow.xaml	1	0
ioSender XL/ioSender XL/MainWindow.xaml.cs	5	0
ioSender/ioSender/MainWindow.xaml	1	0
ioSender/ioSender/MainWindow.xaml.cs	5	0

File > Load Folder reads a folder of per-operation files named `<seq>_<name>_T<tool>.nc` (as produced by the companion Fusion 360 add-in) and assembles them in memory into a single program:

- strips each file's header and program-end footer;
- inserts a `G53 G0 Z0` safe-Z retract + `M6 T<n>` tool change before each toolpath, but only when the file does not already contain an `M6` (so any post works);
- optionally restores rapid moves that Fusion Personal Use downgraded to `G1`;
- shows the result as a collapsible per-toolpath outline (groups default collapsed, and stay expanded while scrolling);
- adds *Start from this toolpath* and *Run just this toolpath* (the latter bounded to that section), each preceded by the `G90 G94 / G17 / G21` prolog.

The new `FusionFolderLoader` is a faithful C# port of the add-in's combine / strip / rapid-restore rules; `FolderPicker` uses the modern `IFileOpenDialog` (`FOS_PICKFOLDERS`) for folder selection. `GCodeBlock` gains a `Section` tag for outline grouping and an optional `AddLineNumbers` gate;

`GrblViewModel` gains `IsFolderView` and a one-shot `RunToBlock` end-bound consumed by `CycleStart`.

Companion Fusion 360 add-in (`ioSenderBatchPost`): a standalone add-in (no external framework) that posts every operation in every setup to its own `<seq>_<name>_T<tool>.nc` file in a chosen folder — the input for Load Folder. Combining, tool-change insertion and rapid restoration are intentionally left to ioSender, so the add-in only posts. A post-processor dropdown (default `grbl.cps`) selects which post Fusion runs per operation. Ships with `install-windows.ps1` / `install-macos.sh` and a README. Fusion auto-discovers it once copied, but it must be enabled once via *Scripts and Add-Ins*.

Note: the `Fusion Addin/` folder is an optional companion tool; the maintainer may prefer to host it separately. It can be dropped from the PR if so.

PR 7 Improve high-DPI / large-text-size layout across the UI

Branch	`pr/highdpi-layout`
Compare	https://github.com/terjeio/ioSender/compare/master...stevenrwood:ioSender:pr/highdpi-layout

File	+	−
CNC Controls Camera/ … /ConfigControl.xaml	3	3
CNC Controls Lathe/ … /ConfigControl.xaml	6	6
CNC Controls Probing/ … /ProbingView.xaml	6	6
CNC Controls Probing/ … /ProbingView.xaml.cs	3	1
CNC Controls/ … /BasicConfigControl.xaml	4	4
CNC Controls/ … /CoordValueSetControl.xaml	6	6
CNC Controls/ … /FeedControl.xaml	5	5
CNC Controls/ … /GotoBaseControl.xaml	5	5
CNC Controls/ … /GotoControl.xaml	1	1
CNC Controls/ … /JogBaseControl.xaml	2	2
CNC Controls/ … /LEDControl.xaml	4	5
CNC Controls/ … /NumericField.xaml	6	3
CNC Controls/ … /NumericField.xaml.cs	3	1
CNC Controls/ … /OutlineBaseControl.xaml	8	4
CNC Controls/ … /OutlineControl.xaml	1	1
CNC Controls/ … /OutlineFlyout.xaml	1	1
CNC Controls/ … /OverrideControl.xaml	3	3
CNC Controls/ … /SpindleControl.xaml	8	8
CNC Controls/ … /StatusControl.xaml	3	3
CNC Controls/ … /StepperCalibrationWizard.xaml	2	2
CNC Controls/ … /StripGCodeConfigControl.xaml	15	9
CNC Controls/ … /ToolView.xaml	2	2
CNC Controls/ … /TrinamicView.xaml	1	1
CNC Controls/ … /WorkParametersControl.xaml	18	18
ioSender XL/ioSender XL/JobView.xaml	6	6
ioSender/ioSender/JobView.xaml	1	1

Makes the UI usable at 125–150% Windows text scaling by replacing fixed control heights/widths with auto-sizing plus `MinWidth`/`MinHeight`, so labels and fields grow with the font instead of clipping or overlapping. Changes are layout-only — no behavioural changes.

- **Shared controls:** the `NumericField` label column now auto-sizes (`MinWidth = ColonAt`) with a `SharedSizeGroup` so labels align in any container that sets `Grid.IsSharedSizeScope`. `CoordValueSetControl`, `StatusControl`, `LEDControl` (dropped a hard `MaxHeight`) and `OverrideControl` no longer pin `Height`/`Width`; the **Spindle** group switches its fixed `Width=250` to `MinWidth` so the RPM field is no longer clipped at larger text sizes.

- **grbl:Settings config controls** (Basic / StripGCode / Lathe / Camera) and both Stepper calibration tabs auto-size their rows and the Axis dropdown; the Probing view auto-sizes its left panel with shared-size label alignment.
- **Main window** (JobView XL + ioSender): right-panel grid / columns / panels auto-size instead of fixed 515 / 250 px, and each panel control (WorkParameters, Spindle, Coolant, Outline, Feed, Goto) auto-sizes.

PR 8 Native Xbox controller support (+ Key Mappings editor polish)

Branch	`pr/xbox-mapping`
Depends on	PR 3 (`pr/keymapping_editor`) — this branch is stacked on it; the figures below are the delta on top of PR 3.
Compare	https://github.com/stevenrwood/ioSender/compare/pr/keymapping_editor...pr/xbox-mapping

File	+	−
CNC Core/CNC Core/XInput.cs	160	0
CNC Core/CNC Core/ControllerService.cs	186	0
CNC Core/CNC Core/ControllerMapper.cs	408	0
CNC Core/CNC Core/CNC Core.csproj	3	0
CNC Core/CNC Core/GrblViewModel.cs	16	0
CNC Core/CNC Core/KeypressHandler.cs	23	0
CNC Controls/CNC Controls/JogBaseControl.xaml.cs	5	0
CNC Controls/CNC Controls/KeyMapEditor.xaml	166	22
CNC Controls/CNC Controls/KeyMapEditor.xaml.cs	356	4

Adds first-class game-controller support with **zero new dependencies** — the project uses no NuGet, so the controller is read via native **XInput** P/Invoke (`xinput1_4.dll`, falling back to `xinput9_1_0.dll`). A ~60 Hz `DispatcherTimer` poll layer runs in parallel to the keyboard handler.

- **Interop / service:** `XInput.cs` wraps the gamepad state and vibration structs; `ControllerService.cs` scans all four slots, edge-detects button down/up, and exposes Connected/Disconnected events, deadzone/normalize helpers and rumble.
- **Button map:** `ControllerMapper.cs` dispatches a default, fully re-mappable button map to machine actions (A = Cycle Start, B = Feed Hold, X = Spindle Stop, Y = Home, Back = Reset, Start = Unlock, D-pad = step-jog X/Y). Actions route through the same `GrblViewModel.ExecuteCommand` path as the on-screen buttons and are gated to a live connection in Idle/Jog/Tool states (with a status message when ignored). A short rumble confirms connection.
- **Analog jog:** the left stick proportionally jogs X/Y and the triggers jog Z, via throttled overlapping `$J=` moves (jog-cancel on release) so motion stays smooth; speed scales with stick deflection. Enable, left-stick deadzone, max-feed multiple of the Jog panel feed, and per-axis invert are user-configurable. The button map and analog settings persist to `ControllerMap.xml`.
- **Controller tab** in the Key Mappings editor: live press indicator, per-button action dropdowns (each button's default marked with a leading `*`), *Restore Defaults* (enabled only when modified), machine-direction-aware tooltips, and the analog-jog settings — machine dispatch is paused while the dialog is open. New `JogDistanceProvider`/`JogFeedProvider` hooks let controller jog match the on-screen Jog panel's current step and feed.

The same branch also carries the keyboard-tab polish (changed-binding highlight with per-row/group reset, *Clear* moved into the row menu, column sizing, and the grouped outline now keeps a section expanded while scrolling instead of collapsing it). The branch is fully LF-normalized, so the counts above are the real diff — no line-ending noise.

PR 9 Configurable main page + flyouts + jog panels (ioSender XL)

Branch	`pr/main-page-layout`
Depends on	The integrated Key Mappings + Xbox base in `master` (PRs 3 & 8); the figures below are the delta on top of that base.
Compare	https://github.com/stevenrwood/ioSender/compare/master...pr/main-page-layout

File	+	−
CNC Controls/CNC Controls/AppConfig.cs	119	20
CNC Controls/CNC Controls/AppConfigView.xaml	2	1
CNC Controls/CNC Controls/AppConfigView.xaml.cs	152	0
CNC Controls/CNC Controls/BasicConfigControl.xaml	2	0
CNC Controls/CNC Controls/CNC Controls.csproj	43	0
CNC Controls/CNC Controls/JogBaseControl.xaml	2	2
CNC Controls/CNC Controls/JogBaseControl.xaml.cs	90	2
CNC Controls/CNC Controls/JogSlider.xaml	66	0
CNC Controls/CNC Controls/JogSlider.xaml.cs	51	0
CNC Controls/CNC Controls/KeyboardJoggingControl.xaml	25	0
CNC Controls/CNC Controls/KeyboardJoggingControl.xaml.cs	34	0
CNC Controls/CNC Controls/MainPageEditor.xaml	61	0
CNC Controls/CNC Controls/MainPageEditor.xaml.cs	146	0
CNC Controls/CNC Controls/MainPanelRegistry.cs	100	0
CNC Controls/CNC Controls/NumericField.xaml	11	1
CNC Controls/CNC Controls/NumericField.xaml.cs	38	0
CNC Controls/CNC Controls/OffsetFlyout.xaml	41	0
CNC Controls/CNC Controls/OffsetFlyout.xaml.cs	114	0
CNC Controls/CNC Controls/PanelFlyout.xaml	43	0
CNC Controls/CNC Controls/PanelFlyout.xaml.cs	80	0
CNC Controls/CNC Controls/SidebarItem.cs	23	4
CNC Controls/CNC Controls/UIJoggingControl.xaml	44	0
CNC Controls/CNC Controls/UIJoggingControl.xaml.cs	29	0
CNC Core/CNC Core/KeypressHandler.cs	22	4
CNC Core/CNC Core/SerialStream.cs	1	1
ioSender XL/ioSender XL/JobView.xaml	8	6
ioSender XL/ioSender XL/JobView.xaml.cs	45	3
ioSender XL/ioSender XL/MainWindow.xaml	3	3
ioSender XL/ioSender XL/MainWindow.xaml.cs	72	5

ioSender XL only. Makes the crowded right-hand panel area configurable instead of fixed. A new **Edit Main Page** dialog (beside *Edit Key Bindings*) is a shuttle editor with three buckets — *Available*, *Main page* (up to six slots), and *Flyouts* — so each named panel can be placed on the main page or as a pinnable right-edge flyout, or left off entirely to reclaim space. Nothing is auto-assigned.

- **Flyouts** share a flush, tab-height border and distribute their content vertically; each has a pin (stays open when another flyout opens, and persists across launches) and a close button.
- **Per-offset flyouts** (G28 / G30 / G54..G59.3 / G92) are tiny *Go* buttons (tooltip shows the live coordinates; G28/G30 also get a *Set* = `G28.1/G30.1`) so several can be pinned beside the jog panel.
- **Jog panels** — UI jogging (distance + speed) and keyboard default-speed — become assignable panels rendered as compact sliders with a read-only value; Settings:App stays the editor. The jog-arrow radios are hidden only when the UI jog panel is assigned.
- **Settings:App** gains opt-in autosave (with a discard prompt) and a *Reset to default* context-menu entry on numeric fields; the old *Edit Key Mappings* button is renamed **Edit Key Bindings**.

Layout choices persist (panels / flyouts / pins serialized as CSV to sidestep the `XmlSerializer` list-append quirk). Also includes robustness fixes: a resilient `SerialStream.OutCount` that no longer throws on a dropped port, and a deferred main-panel build so startup can't race the config load.

PR 10 Macro manager + run-only flyout

Branch	`pr/macro-editor`
Depends on	The integrated base in `master` (the button sits next to PR 3's "Edit Key Bindings" on Settings:App); the figures below are the delta on top.
Compare	https://github.com/stevenrwood/ioSender/compare/master...pr/macro-editor

File	+	−
CNC Controls/CNC Controls/MacroManagerDialog.xaml.cs	308	0
CNC Controls/CNC Controls/MacroManagerDialog.xaml	81	0
CNC Controls/CNC Controls/MacroExecuteControl.xaml.cs	19	8
CNC Controls/CNC Controls/AppConfig.cs	19	0
CNC Controls/CNC Controls/AppConfigView.xaml.cs	10	0
CNC Controls/CNC Controls/MacroExecuteControl.xaml	8	16
CNC Controls/CNC Controls/CNC Controls.csproj	7	0
CNC Core/CNC Core/GCode.cs	5	0
CNC Controls/CNC Controls/JobControl.xaml.cs	3	2
CNC Controls/CNC Controls/MacroToolbarControl.xaml.cs	3	0
CNC Controls/CNC Controls/AppConfigView.xaml	1	0

Adds an **Edit Macros** button to the Settings:App page opening a macro manager — a DataGrid with one row per macro, in the spirit of Fusion's *Scripts and Add-Ins* dialog — and reworks the on-screen macro flyout for running.

- **Name** and a **Prompt** (confirm-before-run) flag are edited in-line; an assignable **Key** column (F1–F12) sets the function key (keys are unique; *Create* auto-assigns the first free one); a **File** column shows the referenced file for `@<path>` macros (full path on hover), blank for inline.
- **Edit** edits the macro's definition (inline G-code, or the `@<path>` reference line) and reads it back; **View** opens what it points to — the referenced file for an `@<path>` macro (created if missing, to author a new one), otherwise the code. **Create**/**Delete** add and remove rows.
- The **macro flyout** is now run-only: a macro-name dropdown plus a **Run** button (the Edit button is gone — editing lives in the manager). It pre-selects the most recently run macro, persisted in `Config.LastMacroId` and updated from every run path (flyout, toolbar, F-keys), falling back to the first.

The macro's function key is decoupled from its `Id` via a new `Macro.FKey` field (legacy configs migrated on load: `Id n` keeps `Fn`). The `@<path>` *run-time* resolution — loading and running the referenced file, with directive processing — is provided by **PR 11**; this PR adds the manager/flyout support for authoring, viewing, and repointing references.

PR 11 Macro Processor Enhancements

Branch	`pr/macro-processor`
Depends on	Off `master`, independent of PR 10. The two are complementary: PR 10's manager *authors* macros, this *executes* them.
Compare	https://github.com/stevenrwood/ioSender/compare/master...pr/macro-processor

File	+	−
CNC Controls/CNC Controls/MacroProcessor.cs	582	0
CNC Controls/CNC Controls/MacroToolbarControl.xaml.cs	1	1
CNC Controls/CNC Controls/MacroExecuteControl.xaml.cs	1	1
CNC Controls/CNC Controls/JobControl.xaml.cs	1	2
CNC Controls/CNC Controls/CNC Controls.csproj	1	0
CNC Core/CNC Core/GCodeParser.cs	31	12
CNC Core/CNC Core/NGCExpr.cs	22	0
CNC Core/CNC Core/Grbl.cs	11	0
CNC Core/CNC Core/GrblViewModel.cs	9	2

A sender-side **macro pre-processor** (`MacroProcessor.Run`) that interprets parenthesised directives before the remaining lines are streamed to the controller. All macro entry points — the top toolbar, the on-screen macro flyout, and the F-key handler — now route through it (previously the toolbar called `ExecuteMacro` directly and silently ignored every directive).

Directives, each a no-op the controller would treat as a comment, interpreted by the sender instead:

- **`(PREREQ, cond, …)`** — require machine state *before* anything streams; if unmet the macro aborts with a message and nothing is sent. Conditions: `homed`, `idle`, `tlo`, `noalarm`, `connected`; stored positions / work offsets `g28`, `g30`, `g92`, `g54`–`g59.3` ("set" if any axis offset is non-zero, read fresh from `$#`); and any other token is required to be a controller `$I` build option (`NEWOPT`, e.g. `EXPR`), matched exactly and case-sensitively. `homed` reads grblHAL's live `sys.homed.mask` from `$#` rather than the change-based status `H:` cache, which can stay stale after a position-loss alarm.
- **`(MBOX, [buttons,] message)`** — modal hold; `OK`/`OKCANCEL`/`YESNO`, Cancel/No aborts the rest.
- **`(WAITIDLE)`** — hold streaming until the controller finishes what was sent and returns to Idle — needed after a controller-side job such as `$F=<file>`, which acks immediately and drops sender input while it runs.
- **`(PROMPT param, default [, label])`** — collect input values in a single up-front dialog and bind them two ways: assign the globals on the controller (so `$F=` files can read them) and substitute the references in the streamed body.
- **`@<path>`** — a macro whose body is a single `@<path>` line is a reference to an external file: that file's current contents are loaded and run in its place, re-read on every run — so a macro can be developed by editing the file directly. The loaded contents go through the same directive pipeline. (PR 10 adds the manager/flyout support for authoring and repointing these references.)

Supporting core changes so macros stream cleanly to an NGC-capable controller, all in the spirit of "don't reject what the controller can evaluate": `ExecuteMacro` now inherits the controller's expression

support (as file streaming already did) and forwards verbatim any line its own parser cannot handle; `GCodeParser` recognises **named O-word subroutine calls** (`O<name> CALL […]`) and forwards them to the controller, which resolves the named sub from its filesystem; and the `$# [HOME:…:mask]` homed-axes mask is parsed for the authoritative `homed` check.

PR 12 Onboarding tooltips

Branch	`pr/onboarding-tooltips`
Depends on	Off `master`, independent. Aimed at users new to CNC.
Compare	https://github.com/stevenrwood/ioSender/compare/master...pr/onboarding-tooltips

File	+	−
`CNC Controls/CNC Controls/Converters.cs`	50	0
`CNC Controls/CNC Controls/StatusControl.xaml`	3	0
`CNC Controls/CNC Controls/StripGCodeConfigControl.xaml`	10	5
`CNC Controls/CNC Controls/JogUiConfigControl.xaml`	10	9
`CNC Controls/CNC Controls/JogConfigControl.xaml`	7	7
`CNC Controls/CNC Controls/BasicConfigControl.xaml`	3	3

Tooltips aimed at users new to CNC, where ioSender's UI is otherwise terse.

- **State field** — a context-aware tooltip explains the current controller state in plain language (Idle, Run, Hold, Check, Tool, Door, …). For an **alarm** it appends the controller's specific message — e.g. `ALARM 4: Probe fail …`, pulled from the `GrblAlarms` enumeration loaded at connect — plus how to recover, so a bare `ALARM:4` is self-explanatory instead of a code to look up. (A new `GrblStateToTooltipConverter` drives a `ToolTip` binding on the State box.)
- **Settings:App** — a "what it does and why it's useful" tooltip on each application setting (the grbl settings page already self-documents in its right-hand pane, so it's left alone). Keyboard jogging (fast/slow/step feed & distance, link-step-to-UI), UI jogging (the four distance and four speed presets, noting distance and speed are selected *independently*), and GCode command stripping (M6/M7/M8/G61-G64, for machines lacking the matching hardware). Also fixes the "Keep MDI focus" tooltip (it had been copy-pasted from the buffering option) and the circular "Send comments" one.

PR 13 Build documentation: Mega-XL upgrade notes + repeatable tool-change guide

Branch	`pr/build-docs`
Depends on	Off `master`, independent. Documentation only — no code or build changes.
Compare	https://github.com/stevenrwood/ioSender/compare/master...pr/build-docs

File	+	−
`docs/Mega-XL-Upgrades.html`	840	0
`docs/Mega-XL-Upgrades.pdf`	bin	0
`docs/Repeatable-Tool-Change.html`	419	0
`docs/Repeatable-Tool-Change.pdf`	bin	0

Two self-contained HTML documents (each rendered to a companion PDF), added under a new `docs/` folder. Purely additive; touches no source.

- **Repeatable-Tool-Change** — a turnkey, novice-friendly tool-change guide built around a **G59.3 fixed toolsetter** and a `T<n>/M6` flow, with a top-down machine travel diagram (color-coded paths). Aimed at the goal of repeatable, hands-off tool changes for users new to CNC.
- **Mega-XL-Upgrades** — build notes for an upgraded Kickstarter v1.0 Mega V XL: frame/spindle/motion/probing/power sections, a back-of-enclosure two-panel harness model (hand-built SVG diagrams), a cable & wiring schedule, and an **interactive bill-of-materials worksheet** — an in-page editor (double-click to edit Qty / Unit / Description / Source, live extended & category subtotals & grand total, URL shortening, Save-to-download) plus a linked table of contents.

The Mega-XL notes are specific to one machine and may be of limited upstream interest — they can be hosted separately or dropped from the PR. The repeatable tool-change guide is more generally applicable. The associated controller macros (`tc.macro`, `probe_tfl.macro`) and the ATC provisioning UI are intentionally *not* in this PR — they remain with the macro work, pending firmware support.

PR 14 Combined SD + LittleFS file browser on the SD Card tab

Branch	`pr/sdcard-filesystems`
Depends on	Off `master`, independent.
Compare	https://github.com/stevenrwood/ioSender/compare/master...pr/sdcard-filesystems

File	+	−
`CNC Controls/CNC Controls/GrblFilesystems.cs`	142	0
`CNC Controls/CNC Controls/SDCardView.xaml.cs`	136	17
`CNC Controls/CNC Controls/SDCardView.xaml`	4	2
`CNC Controls/CNC Controls/CNC Controls.csproj`	1	0
`tests/GrblFilesystemsTests.cs`	83	0
`tests/README.md`	22	0

The SD Card tab listed only one filesystem (the SD card, or whatever was the current filesystem). On grblHAL builds that have both an SD card and a LittleFS store, the other filesystem — and any macros on it — were invisible.

This enumerates every mounted filesystem via `$FI` (`[FS:<name>@<path> size, free]`) and lists them together in one grid with a new **Location** column, plus a per-filesystem **free-space banner** above the list. *Run / Download and run / Delete* now target the file's own filesystem — a bare name on the root volume (unchanged single-SD behaviour) or an absolute path on a sub-mount such as `/littlefs`.

- **Backward compatible:** builds that don't implement `$FI` (or report nothing mounted, `error:65`) return no `[FS:…]` lines, and the view falls back to the original single-filesystem mount + `$F` listing untouched. An SD card with files in its root works exactly as before, now also showing free space.
- **New `GrblFilesystems.cs`** holds pure, dependency-free helpers (mount-line parser, human-readable size, path qualification, free-space summary) and the `FsMount` model; the live `$FI`/`$F` querying lives in `GrblSDCard`.
- **Unit tested:** `tests/GrblFilesystemsTests.cs` exercises the pure helpers (29 assertions, run via the Framework `csc` — see `tests/README.md`; the app has no test project).

Builds clean (CNC Controls and the full ioSender solution). Groundwork the paused ATC macro-provisioning work can reuse to target the `$FI`-discovered LittleFS path instead of a hardcoded one.

PR 15 USB / Ethernet / simulator connection support + Connect menu

Branch	`pr/connect-simulator`
Depends on	Off `master`, independent.
Compare	https://github.com/stevenrwood/ioSender/compare/master...pr/connect-simulator

File	+	−
CNC Controls/CNC Controls/SimulatorManager.cs	232	0
CNC Controls/CNC Controls/AppConfig.cs	134	28
CNC Controls/CNC Controls/PortDialog.xaml.cs	131	2
ioSender XL/ioSender XL/MainWindow.xaml.cs	114	14
CNC Controls/CNC Controls/PortDialog.xaml	68	6
ioSender XL/ioSender XL/MainWindow.xaml	30	2
CNC Core/CNC Core/TelnetStream.cs	14	3
ioSender XL/ioSender XL/ioSender XL.csproj	12	0
CNC Controls/CNC Controls/UIUtils.cs	12	4
ioSender XL/ioSender XL/JobView.xaml.cs	8	0
CNC Core/CNC Core/HelperClasses.cs	8	2
CNC Core/CNC Core/GrblViewModel.cs	7	0
simulator/README.md	5	0
readme.md	2	0
CNC Controls/CNC Controls/Widget.cs	1	1
CNC Controls/CNC Controls/CNC Controls.csproj	1	0

Adds first-class connection setup to the sender: pick USB (serial), Ethernet (`host:port`, now accepting hostnames as well as IPv4), or a local grblHAL **simulator** from a new tab in the connection dialog, and (re)connect at any time from a menu.

- **Simulator tab:** start/stop a bundled grblHAL simulator (launched minimized, window titled by the executable name, with a confirmation), then connect to `127.0.0.1:<port>`. The executable is located in a `simulator` sub-folder next to the app (a build step copies it there from the repo; binaries are not committed). Right-clicking the **Target** status item offers a *Validate controller* action (a stub here that launches `grblHAL_validator`; replaced by the in-app validator in PR 16).
- **Connect / Reconnect:** a `File > Connect…` item opens the dialog; while connected it becomes *Reconnect…* (disconnects, then re-opens the dialog so a different target can be chosen). The app now stays open when started without a connection instead of exiting, so the menu is reachable.
- **Status bar** shows the current connection target (`GrblViewModel.ConnectionTarget`), now coloured green when connected and red when not.
- **Auto-start on reconnect:** when the saved target is the simulator the app launches the bundled simulator before connecting (silently — no confirmation popup), so a startup auto-reconnect to a simulator target brings it up automatically. The choice is remembered (a `StartSimulator` setting), since a `127.0.0.1:<port>` target is otherwise indistinguishable from a real network controller.

- **Startup ordering:** `AppConfig.SetupAndOpen` is split into `LoadConfig` (run synchronously at construction, so config is available before any control's `Loaded` handler reads it) and `OpenConnection` (deferred to `ApplicationIdle` so the main window paints before the — possibly blocking — connection dialog). Transport is chosen by target string (`COMx` vs `host:port`) rather than a leading-digit IPv4 check.
- **Simulator robustness & speed:** `SimulatorManager` reaps any stale instances of the managed simulator before launching (stopping the debugger bypasses the exit cleanup, leaving a zombie that holds the single accept slot and hangs the next connect). A `SimulatorSpeedup` setting passes the simulator's `-t` pacing flag (1 = real-machine speed, 0 = as-fast-as-host). And `TelnetStream.ReadComplete` captures the stream into a local and bails when closing, so an intentional `Close()` races to a clean stop instead of an NRE on the in-flight async read.
- **"My Machine" profile:** `SimulatorManager.BuildMyMachineEeprom` can mirror a real controller's current `$` settings into a private `MyMachine.DAT` EEPROM image by driving a throwaway headless simulator and replaying the settings into it. The Simulator tab then offers a *My Machine* option that boots the simulator against that image, so connecting to the simulator reproduces the real machine's context (travel, homing, axis setup) rather than the generic default.

Builds clean (CNC Controls and the full ioSender solution) off `master`. The simulator / validator binaries come from the grblHAL Simulator build artifacts and are not committed (see `simulator/README.md`).

PR 16 Validate controller (check-mode g-code conformance test)

Branch	`pr/validate`
Depends on	Stacked on `pr/connect-simulator` (PR 15).
Compare	https://github.com/stevenrwood/ioSender/compare/pr/connect-simulator...pr/validate

File	+	−
`CNC Controls/CNC Controls/ValidateProcessor.cs`	1091	0
`CNC GCodeViewer/CNC GCodeViewer/Renderer.xaml.cs`	81	16
`ioSender XL/ioSender XL/MainWindow.xaml.cs`	3	21
`CNC Controls/CNC Controls/CNC Controls.csproj`	1	0

Replaces the placeholder *Validate controller* action (PR 15) with a real, in-app conformance test: it streams a g-code program tailored to the connected controller's reported capabilities and reports which commands it accepts — without moving the machine.

- **Capability-tailored test:** generated from `$I` build options, axis count, `$32` mode, tool count, aux-I/O counts and the work coordinate systems present, so a feature the firmware was never built with is not tested (and cannot be reported as a false failure).
- **Check mode, no motion:** streamed in `$C` check mode and parsed only — the machine never moves, so homing and a clear envelope are not required. One feature per line, so an `error:N` pinpoints exactly which command the controller rejected.
- **Robust against grbl's check-mode behaviour:** a check-mode error *latches* the parser (every later line then errors), so the run recovers on each error — reset, re-enter check mode (unlocking first if the reset re-alarmed a homed machine), re-apply the modal set-up — then resumes, always re-confirming check mode before sending another line so a test line can never reach the controller as real motion.
- **Non-destructive:** check mode *commits* `G10`/`G92` writes, so the run snapshots and restores the work offsets / tool table and avoids the un-restorable writes (`G28.1`/`G30.1`, and `G92` when an offset is active). Machine settings (`$$`) are never written.
- **Progress & results:** a small non-modal panel shows a live pass/fail tally and the current test as it streams; on completion a *View Summary* button opens the full report (grouped by category, error code/message per failure, the pre-run parameter snapshot, Copy-to-clipboard). The controller is left in `Idle` with check mode off.
- **3D visualization:** the passed bounded-motion features are assembled into a runnable program (absolute `G90` moves at a fast feed) and loaded into the buffer, so the user can press Cycle Start and watch the toolpath run in the 3D view via the normal job path. The viewer also gains an *always-active machine scene* (work envelope, axes and a live tool cone shown when homed even with no program loaded), plus fixes to the tool/executed-trail animation subscription and a bounded executed-trail advance (an out-of-range block was an unbounded spin).

Right-click the **Target** status item → *Validate controller*. Builds clean on `pr/connect-simulator`; the only failures on a stock grblHAL are commands it genuinely does not implement (e.g. `G90.1`, `G61.1`, `G64`, `M52`; `M56` needs parking-override enabled).

PR 17 Restore main window position across launches

Branch	`pr/window-restore`
Depends on	Stacked on `pr/connect-simulator` (PR 15) — the restore runs in its `CompleteStartup`.
Compare	https://github.com/stevenrwood/ioSender/compare/pr/connect-simulator...pr/window-restore

File	+	−
`ioSender XL/ioSender XL/MainWindow.xaml.cs`	58	7
`CNC Controls/CNC Controls/AppConfig.cs`	2	0

The "Restore last window size on startup" option restored only the window *size*; the position was discarded (clamped to the top-left). This persists the position too.

- **Position persisted:** new `WindowLeft`/`WindowTop` settings, saved on close (using `RestoreBounds` when maximized so the un-maximized rectangle is stored) and restored on startup — but only when the saved rectangle lands on a currently-connected monitor (checked against the whole virtual desktop), so a window saved on a since-removed screen can't open off-screen.
- **Takes effect immediately:** the live "save" flag was only set at startup, so toggling the option mid-session did nothing until a restart. It now updates live and captures + persists the current placement the instant the option is enabled.

Builds clean on `pr/connect-simulator`. Stacked there because the size/position restore lives in that branch's `CompleteStartup`; the underlying `KeepWindowSize` option is from `master`.

PR 18 grbl:Settings — highlight changed settings + revert

Branch	`pr/grbl-settings`
Depends on	Off `master`, independent.
Compare	https://github.com/stevenrwood/ioSender/compare/master...pr/grbl-settings

File	+	−
`CNC Core/CNC Core/Grbl.cs`	65	2
`CNC Controls/CNC Controls/GrblConfigControl.xaml`	40	5
`CNC Controls/CNC Controls/GrblConfigControl.xaml.cs`	32	0

Orca/PrusaSlicer-style indication of which controller $ settings you have changed, on the grbl:Settings **Basic** tab. Settings whose value differs from the value read at session start — and any setting group that contains one — are shown bold/orange, so changes are visible at a glance across the whole tree. Applies to both the grblHAL settings tree and the classic-grbl grid.

- **Revert from a right-click menu:** *Revert to value at startup* on a setting, and *Revert all in group to value at startup* on a group node.
- **Three baselines, cleanly separated:** `GrblSettingDetails` now tracks the current value, the frozen *startup* value (drives the `IsModified` highlight) and the *loaded* controller value (drives the `IsDirty` needs-write state). Making `IsDirty` mean "differs from the controller" rather than "changed" fixes a spurious "settings changed, save now?" prompt when reverting an edit back to the controller value.
- **Live group highlight:** `GrblSettingGroup` implements `INotifyPropertyChanged` so a group's marker updates as its settings change.

Builds clean off `master` (CNC Controls and the full ioSender solution); verified against the bundled grblHAL simulator. Per-setting "revert to factory default" is intentionally deferred — grblHAL exposes no per-setting default (only the global $RST=$ reset-all), so factory revert would need a future firmware-side snapshot of as-flashed values.

PR 19 Console — inline terminal-style command prompt

Branch	`pr/mdi-console`
Depends on	Off `master`, independent.
Compare	https://github.com/stevenrwood/ioSender/compare/master...pr/mdi-console

File	+	−
CNC Controls/CNC Controls/ConsoleControl.xaml.cs	82	0
CNC Core/CNC Core/GrblViewModel.cs	20	0
CNC Controls/CNC Controls/ConsoleControl.xaml	16	2
CNC Core/CNC Core/KeypressHandler.cs	7	0
CNC Controls/CNC Controls/MDIControl.xaml.cs	5	16
CNC Controls/CNC Controls/MDIControl.xaml	1	1

Makes the **Console** tab read like a traditional terminal by adding a command input line at the bottom, while keeping the global single-line MDI strip for the other tabs (3D View, Program). A natural answer to "why is MDI a separate strip instead of a console prompt?" — now it can be both.

- **Reuses the MDI plumbing:** Enter routes the line through the existing `MDICommand`, which already echoes the command into the console's `ResponseLog` — so the typed command and its response read inline like a shell. No `JobView` changes.
- **Shared command history:** a single `GrblViewModel.MDIHistory` (newest first) is fed by both the MDI strip and the console prompt and surfaced in both — the strip's dropdown binds to it and the prompt recalls it with Up/Down (Esc clears). A command entered in either place is recallable in the other. (The view-model `CommandLog` is not populated for these entries, so it is not used.)
- **Same guards, auto-focus:** the prompt is locked out while a job runs (same as the MDI strip) and grabs focus when the console becomes visible (tab switch or the pop-out console window) so it's ready to type into.
- **Pop-out for free:** the prompt is self-contained in `ConsoleControl`, so the detachable console window (which hosts the same control) gets it too.
- **No jog conflict:** the keyboard-jog handler jogs whenever a key is *mapped* to jog (Up/Down are by default) *before* its per-key text-box guard, so the prompt's arrows would otherwise move the machine. `ProcessKeypress` now forces no-jog when the focused element is a `TextBox` tagged `"MDI"`, and the prompt is tagged accordingly — uniformly exempting both the MDI strip's edit box and the console prompt.

Builds clean off `master` (CNC Controls and the full ioSender solution); verified against the bundled grblHAL simulator: commands echo + execute, auto-focus, history recall (Up/Down step through prior commands), and an A/B test confirming Up jogs the DRO when a non-input element is focused but does not when the console prompt is focused. Keys are handled in `PreviewKeyDown` so the arrows reach the prompt rather than being consumed by the TextBox.

PR 20 3D view — Ctrl+click to jog + per-axis orientation

Branch	`pr/click-to-jog`
Depends on	Off `master`, independent.
Compare	master...pr/click-to-jog

File	+	−
`CNC GCodeViewer/CNC GCodeViewer/Renderer.xaml.cs`	164	52
`CNC Core/CNC Core/Grbl.cs`	26	1
`CNC GCodeViewer/CNC GCodeViewer/RenderControl.xaml`	11	7
`CNC Core/CNC Core/GCodeEmulator.cs`	3	0
`CNC Controls/CNC Controls/AppConfig.cs`	2	0

Ctrl+left-click in the 3D or top-down (XY) view jogs the machine to the clicked XY position — a quick coarse-positioning aid that complements the jog buttons (park over a corner to set work zero, check alignment, touch-screen rigs).

- **Safe by construction:** the move always retracts **Z to the top of travel first**, then rapids to the new XY, so the traverse can never plough through stock or fixtures. Only X/Y move; plain left-drag still rotates the camera.
- **How it maps:** the click ray is intersected with the work Z=0 plane (HelixToolkit `UnProject`) to recover an XY target, converted to machine coordinates (`MPos = WPos + WCO`), clamped to the soft-limit travel exactly as the jog limiter does, and emitted as two queued `$J=G53` jogs at the controller's max rate (`$110`–`$112`) so it moves at rapid speed.
- **Guarded:** requires homed + idle + max travel set + soft limits (`$20`) on, with a status-bar reason when a prerequisite is missing. Gated by a `GCodeViewerConfig.ClickToJog` setting (default on).
- **Orientation fix (prerequisite):** the rendered machine frame — work envelope, floor grid and crosshair — now honors the `$23` homing-direction mask + force-set-origin *per axis*, so the view matches any home corner (e.g. top-back-left, X+/Y−/Z−) instead of assuming +X/+Y. Standard +X/+Y/−Z machines render identically to before. `GrblInfo.ForceSetOrigin` now reads the live `$22` bit 3 rather than the `$I` `OPT` 'Z' flag (absent on some builds, e.g. the simulator), so the orientation is correct even when the option string omits it. The camera bounding box in the always-on machine scene is derived from the same per-axis envelope, fixing a mirrored home corner in the 3D view.
- **Toolbar tidy-up:** the 3D-view toolbar tooltips now show on disabled buttons and are reworded for consistency, the *save view* button is no longer needlessly gated, and the *show job envelope* button enables only with a file loaded. A null-tokens guard in `GCodeEmulator` avoids a crash when the machine scene renders with no program loaded.

Builds clean off `master` (CNC GCodeViewer chain). Verified on the author's machine: Ctrl+click retracts Z then rapids to the clicked XY at the configured max rate, and the envelope/grid orient to the real home corner.

PR 21 grbl:Settings — Machine Setup Wizard

Branch	`pr/machine-setup-wizard`
Depends on	Off `master`, independent.
Compare	master...pr/machine-setup-wizard

File	+	−
CNC Controls/CNC Controls/MachineSetupWizard.xaml.cs	682	0
CNC Controls/CNC Controls/MachineSetupWizard.xaml	273	0
CNC Controls/CNC Controls/MachineCatalog.cs	194	0
CNC Controls/CNC Controls/GrblConfigView.xaml.cs	30	0
CNC Controls/CNC Controls/CNC Controls.csproj	8	0
CNC Controls/CNC Controls/GrblConfigView.xaml	3	0
CNC Controls/CNC Controls/AppConfig.cs	2	0
CNC Controls/CNC Controls/IGrblConfigTab.cs	2	1

A guided, first-run **Machine Setup Wizard** tab in grbl:Settings that configures the machine-description settings a fresh controller needs — so a new machine can be described without hunting through the `$` list. Auto-selected when the machine looks unconfigured (no remembered machine, or `$130–$132` all zero).

- **Single page, nothing written until applied:** the whole machine description is laid out on one page — machine selector, a clickable home-position picture, a per-axis table, and the remaining homing & limits questions — so it reads at a glance instead of an eight-step click-through. *Apply* writes the changes via `GrblSettings.Save()`.
- **Machine catalog:** a built-in catalog (`MachineCatalog.cs`) of common hobby machines as manufacturer → product → model drop-downs (or a custom X×Y), each carrying a sensible home corner and force-set-origin default; the first entry is a generic XYZ machine with limit switches. Picking a model pre-fills the page, which the user then confirms or tweaks.
- **Axis table:** an editable per-axis grid — travel limit (`$130–$132`), max rate (`$110–$112`), steps/mm (`$100–$102`), invert direction (`$3`) and normally-closed limit (`$5`) — edited in place.
- **Home corner:** a clickable isometric box — click the corner where the machine homes — sets the X/Y homing directions (`$23`) and auto-enables force-set-origin (`$22` bit 3) on grblHAL; Z is fixed to the top of the gantry. This is the same convention that orients the 3D view (PR 20).
- **Homing & limits:** hard limits (`$21`) default on, with the remaining global questions as a short list and the explanatory help moved into per-input tooltips. A right-justified *Firmware Info* button shows the live `$I` build output in a non-modal dialog.
- **Remembered & forgettable:** the chosen machine is saved (`AppConfig.LastMachine`) and restored next launch — keeping the live controller values over the catalog preset on restore — and a *Forget machine* button clears it so the wizard reappears next time.

Builds clean off `master` (CNC Controls and the full ioSender solution). UI verified by the author against a live controller and the bundled simulator; interactive `$3` direction-verify and homing-cycle-order editing were intentionally left out of this first cut. The first-run gate that forces the wizard to the foreground until a machine is applied, and the "copy this machine into the simulator's My Machine profile" action, depend on connection-side plumbing and ride along in the integration branch rather than this stand-alone PR.

PR 22 Keyboard jog — keep active when Job-view focus drifts

Branch	`pr/keyboard-jog-focus`
Depends on	Off `master`, independent.
Compare	master...pr/keyboard-jog-focus

File	+	−
`ioSender XL/ioSender XL/MainWindow.xaml.cs`	19	0
`ioSender XL/ioSender XL/JobView.xaml.cs`	4	1

Keyboard jogging only fired through `JobView.OnPreviewKeyDown`, which needs keyboard focus inside the Job view's tree. Interacting with a flyout, side panel or menu moves focus out, so the arrow keys stopped jogging until the view was re-focused (e.g. by switching tabs). This keeps jog alive on the Job page wherever focus has drifted.

- **Window-level forwarding:** the existing `MainWindow` tunneling `PreviewKeyDown` (always fires) forwards jog keys to `JobView.ProcessKeyPreview` when the Job view is the current view but is not keyboard-focused. The focused case is unchanged — `JobView` still handles it, including its MDI/DRO/spindle/work-params jog gates.
- **Skips text input:** forwarding is suppressed when focus is in any `TextBoxBase`, so typing in the MDI strip or console prompt never jogs.
- **Clean stop:** a mirrored `PreviewKeyUp` forwards the key-up too, so a continuous jog started while the view was unfocused still receives its release and stops (no text-input guard here — an in-progress jog must always cancel).

Builds clean off `master` (the full ioSender solution). Two-file change reusing the console-shortcut `PreviewKeyDown` hook already on `master`; `ProcessKeyPreview` is promoted to `public` so the window can forward into it.